ESP32-S3 SMART ATTENDANCE SYSTEM

Full-Stack Biometric & RFID Access Controller Project Documentation

1. Executive Summary & Overview

The **ESP32-S3 Smart Attendance System** is a premium, edge-computing embedded device designed to manage employee and student attendance in corporate or educational environments. It combines high-performance microcontroller capabilities with robust physical security protocols, dynamic visual feedback, and comprehensive software administration.

By utilizing an **ESP32-S3 Dual-Core SoC**, the system operates as a stand-alone biometric door controller and logging terminal. It supports contactless card authentication, high-accuracy capacitive fingerprint scanning, and a 2-Factor Authentication (2FA) mode where card swipes are combined with biometric validation. The device features both **Wi-Fi** and **W5500 SPI Ethernet** for communication, an external **64 KB (24C512) EEPROM** for secure local offline log storage, a **3.5" TFT Display with Touch Screen** running a custom state-driven user interface, and a built-in asynchronous HTTP server that hosts a fully compressed dashboard UI for administrative management, reports, user profiles, shift timers, and OTA firmware updates.

2. System Architecture & Block Diagram

The system's modular architecture separates hardware inputs (sensors), feedback mechanisms (TFT, buzzer, LEDs), and external storage and networking interfaces. The shared SPI bus integrates the RFID Reader, Ethernet controller, TFT Display panel, and TFT Touch controller simultaneously, using dedicated chip select lines.

Biometric/RFID Inputs	Processor & Core Logic	Network & Storage
• MFRC522 RFID Reader (SPI) • R503 Fingerprint Sensor (UART)	**ESP32-S3 Xtensa LX7** • Dual Core @ 240MHz • LittleFS Local Flash Storage • 2-Factor Auth State Machine	• W5500 SPI Ethernet (LAN) • 2.4GHz Wi-Fi (AP/STA) • 24C512 EEPROM (I2C Logs)
Visual & Audio Output	**User Interactions**	**Web Dash Management**
• 3.5" TFT Display (ILI9488) • RGB NeoPixel & LEDs • Active Piezo Buzzer	• XPT2046 Touch Screen Panel • Standby Clock & Live Weather • Direction: In / Out selector	• Async Web Server (Port 80) • Restful API & OTA Uploads • Shift & Holiday Configurator

3. Hardware Pinout & Interface Wiring

All peripheral modules are integrated into the ESP32-S3 GPIO layout as follows. The SPI bus operates cooperatively among the display, touch screen, RFID, and Ethernet controllers.

ESP32-S3 GPIO	I/O	Connected Module	Module Pin	Function & Description
GPIO 1	Output	TFT Display	BL	TFT Backlight control line (Active-High)
	Output		Anode (+)	Authorized access success indicator (via 220Ω resistor)
GPIO 3	Output	MFRC522 RFID	RST	Hardware Reset pin to reinitialize the card reader
	Output		Anode (+)	Access denied alert indicator (via 220Ω resistor)
GPIO 5	Output	Active Buzzer	Positive (+)	Audible beep feedback for swipe confirmation
	Output		RST	TFT Reset line to initialize the display
GPIO 7	Output	TFT Display	RS (DC)	TFT Register Select (Data/Command select line)
	Bi-dir		SDA / Pin 5	I2C Serial Data line for offline logs
GPIO 9	Output	24C512 EEPROM	SCL / Pin 6	I2C Serial Clock line for offline logs
	Output		SDA (CS)	Dedicated SPI Chip Select line (Active-Low)
GPIO 11	Output	RFID, ETH & TFT	MOSI	Shared SPI bus Master-Out Slave-In line
	Output		SCK	Shared SPI bus Serial Clock line
GPIO 13	Input	RFID, ETH & Touch	MISO / T_DOUT	Shared SPI bus Master-In Slave-Out / Touch Data Out
	Output		SCS (CS)	Dedicated SPI Chip Select line (Active-Low)
GPIO 15	Output	TFT Display	CS	Dedicated SPI Chip Select line (Active-Low)
	Input		TXD (White)	UART RX2 channel to receive data from sensor
GPIO 17	Output	R503 Fingerprint	RXD (Green)	UART TX2 channel to send commands to sensor
	Output		T_CS	Dedicated SPI Touch Chip Select line (Active-Low)
GPIO 21	Input	TFT Touch	T_IRQ	TFT Touch Interrupt request input line
	Power Out		VCC / VIN	High-power rail for TFT display and backlight
3.3V / 3V3	Power Out	RFID, ETH, R503, EEPROM	VCC / Pin 8	Operating voltage rail for logical circuits
	Ground		GND / Pin 4	Common electrical ground reference for the whole system

4. Power Infrastructure & Electrical Strategy

Integrating high-draw peripherals such as a 3.5" LCD backlight, a capacitive biometrics sensor, and an Ethernet driver demands specific power management measures to avoid system brownouts, resets, or serial packet losses:

- **Transient Current Demands:** The W5500 SPI Ethernet module draws up to 150mA during active network transmissions. The R503 Fingerprint Sensor spikes to 120mA during imaging scans. Additionally, the TFT 3.5" Display Backlight draws up to 150mA. To accommodate these transient current peaks, the system requires a high-quality external power adapter or a USB port capable of delivering at least **1.0A to 1.5A continuous current**. Connecting the device to standard computer USB 2.0 ports (500mA max) may cause immediate brownout resets when multiple modules are activated.
- **Hardware Start-Up Control:** To mitigate inrush current during booting, the TFT backlight (TFT_BL) is wired to **GPIO 1** and is configured as a manual, active-high digital pin. The firmware initializes it ONLY after the core modules, Wi-Fi, and Ethernet

hardware have finished drawing their initial start-up power spikes, preventing bootloop conditions.

- **SPI Bus Clock Segregation:** The display panel and touchscreen share the SPI bus (SPI2_HOST) with the MFRC522 RFID reader and W5500 Ethernet. The Arduino SPI object (used by MFRC522) operates at 4MHz (Mode 0), whereas the display controller (LovyanGFX) operates at 15MHz. To prevent bus conflicts and data corruption, the firmware enforces strict bus locking and reconfigures the SPI parameters before accessing the RFID module.

5. Embedded Software & Firmware Capabilities

A. Non-Blocking TFT State-Driven UI

The firmware implements a non-blocking state machine using the high-performance **LovyanGFX** graphics library. This design ensures that time-consuming tasks (such as searching for Wi-Fi networks, establishing network handshakes, or awaiting fingerprint scans) do not block display redraws or screen transitions. The system state progresses through the following defined modes: *TFT_BOOT*, *TFT_IDLE*, *TFT_PUNCH_SUCCESS*, *TFT_PUNCH_DENIED*, *TFT_DIRECTION_SELECT*, *TFT_WAITING_2FA_FINGER*, *TFT_ENROLL_SCANNING*, and *TFT_ENROLL_CONFIRMED*. Additionally, the touchscreen enables interactive, tactile controls for logging attendance direction (Punch-In vs. Punch-Out).

B. Two-Factor Authentication (2FA) Workflow

For areas requiring enhanced security, the system features a 2FA workflow (Card + Fingerprint). When an employee swipes their RFID Card, the display reads their user profile, transitions to the *TFT_WAITING_2FA_FINGER* screen, and prompts the user to place their finger on the sensor within a 10-second window. Access is granted only when both identifiers match, preventing card-sharing or credential fraud.

C. External EEPROM Logs & Compact Deletion

The system records all transactions locally to the 24C512 external EEPROM, organized as a ring buffer. This design provides physical, non-volatile data persistence that is unaffected by firmware updates or main flash formatting.

Field Name	Data Type	Size	Description / Mapping
`timestamp`	uint32_t	4 Bytes	Unix Epoch timestamp (seconds since 1970)
`uid`	char[12]	12 Bytes	RFID UID string or Fingerprint ID template index
`direction`	uint8_t	1 Byte	Punch Direction: 1 = Punch In (Shift Start), 2 = Punch Out (Shift End)
`status`	uint8_t	1 Byte	Attendance Status: 1=On-Time, 2=Late, 3=Early Exit, 4=Accepted, 5=Denied
`method`	uint8_t	1 Byte	Verification Method: 1 = Fingerprint, 2 = RFID Card, 3 = Manual Web Punch
`padding`	uint8_t	1 Byte	Structure alignment padding (aligns struct size to exactly 20 bytes)

In-Place Date-Based Log Compaction: Since the EEPROM is a raw block memory device, deleting logs for a specific date requires custom wear-leveling management. The firmware copies all logs *excluding* the target date into a temporary file on the local flash storage (LittleFS), clears the EEPROM storage, and writes back the remaining logs sequentially, rewriting the index headers to ensure zero fragmented memory gaps.

D. Web Dashboard, Provisioning Portal & OTA Updates

An asynchronous web server runs on Port 80, serving an administration dashboard. If Wi-Fi credentials are unconfigured or inaccessible, the device initiates a captive portal (AP Mode) at 192.168.4.1. Administrators can download records in CSV format, add/edit users, manage biometric templates, and upload compiled firmware binaries (`.bin`) via an OTA interface to update the system remotely.

E. Dynamic Online Weather Display

When connected to the internet, the device queries meteorological APIs to fetch local parameters (temperature, wind, and humidity). The standby screen displays these values dynamically alongside a custom **vector weather icon** drawn pixel-by-pixel, representing Sunny, Crescent Moon (night), Cloudy, Rainy, Thunderstorm, or Snowy conditions.

6. Web Dashboard HTTP API Reference

The asynchronous web backend exposes a set of RESTful API endpoints used by the dashboard interface:

Endpoint Route	Method	Request Parameters & Functionality Description
`/api/login`	POST	Logs user/admin in, validating credentials stored in auth configuration.
`/list-users`		Retrieves JSON list of all enrolled user profile files in LittleFS.
`/save`	GET	Creates or updates a user profile file (parameters: *uid, name, role, roll*).
`/delete-user`		Deletes profile text file (parameter: *uid*) and wipes associated FP template.
`/get-history`	GET	Queries log history stored in EEPROM and returns JSON for dashboard tables.
`/download-logs`		Generates and streams a downloadable comma-separated values (CSV) log file.
`/delete-logs`	GET	Compact deletion of logs belonging to a specific date (parameter: *date*).
`/start-enroll`		Initiates biometrics sensor recording sequence for a given template ID.
`/set-wifi`	GET	Receives new target SSID and Password and attempts to connect in Station mode.
`/set-shift`		Saves corporate shift work hours (parameters: *start, late, end*) to LittleFS.
`/upload`	POST	Accepts compiled binary file upload and triggers ESP32 OTA flash write.

7. Project Directory Layout & File Manifest

- **Attendance_System.ino:** The main sketch initializing variables, LovyanGFX display classes, hardware interfaces, RFID interrupt handlers, 2FA workflows, and core operation loop.
- **web_routes.ino:** Implements AsyncWebServer callback routes, REST JSON endpoints, network scanning operations, OTA integration, and captive portal redirections.
- **eeprom_logger.ino:** Implements raw I2C block read/write operations, log index headers loading, ring-buffer queue insertions, and date-based database compaction.
- **eeprom_logger.h:** Declares the structured log format structure (`EEPROMLogEntry`), address layout definitions, and public function signatures.
- **html_page_gz.h:** Contains the minified, gzipped, raw hexadecimal byte-array of the HTML, CSS, and JS web dashboard for rapid page delivery.
- **compress_html.py:** A helper script to automatically compress the raw frontend assets, converting them to ESP32-compatible header file formats.

8. System Warnings & Maintenance Guidelines

1. Physical Bus Hardware Pullups: The I2C EEPROM requires hardware
and SCL (GPIO 9) lines. Relying solely on internal ESP32 pull-ups may lea
2. Watchdog Safeguard Configuration: The ESP32 task watchdog (TWD
blocking the main loop for more than 5 seconds will trigger an automatic ha
the watchdog to prevent triggers during flash operations.